

שנקר ביי"ס גבוה להנדסה ולעיצוב  
החוג להנדסת תוכנה  
קורס 'תכנות מערכות' (קורס מספר 3503820)  
תשפ"ו – סמסטר קיץ

מרצה: יצחק נודלר

### **תרגיל בית מס' 1**

**תרגיל תכנות מס' 1: מעתיק קבצים מתקדם מבוסס קריאות מערכת ומדידת זמנים פנימית**

מועד הגשה: 08-09-2026

#### **1. רקע ומטרות התרגיל**

הקורס 'תכנות מערכות' מכניס אתכם אל מאחורי הקלעים של סביבת העבודה של מערכת ההפעלה **Linux (Kernel 6.14)**. בניגוד לקורסים קודמים שבהם השתמשתם בספריות עזר עשירות (כמו stdio ב-C), בתרגיל זה תעבדו ישירות מול הליבה (**Kernel**) באמצעות קריאות מערכת (**System Calls**).

מטרת העל של תרגיל זה היא להקנות לכם הבנה מעמיקה של:

1. האופן שבו מערכת ההפעלה מנהלת קבצים באמצעות טבלת תיאורי קבצים (File Descriptor Table).
2. העלות המערכתית והביצועית של מעבר בין מרחב משתמש (User Space) למרחב הליבה (Kernel Space).
3. כתיבת קוד קלט/פלט חסון (Robust I/O) המתמודד נכון עם שגיאות חלקיות, פסיקות אותות (EINTR), וניהול זיכרון ידני.

#### **2. תיאור המטלה**

עליכם לפתח תוכנית בשפת C הנקראת `my_cp` המבצעת העתקה של קובץ מקור לקובץ יעד. התוכנית תקבל פרמטר שלישי המגדיר את ה- **Granularity** (גודל הבלוק) שבו תשתמש התוכנית בכל מחזור של קריאה וכתיבה.

#### **ממשק שורת הפקודה (CLI Syntax):**

התוכנית תקבל בדיוק שלושה ארגומנטים משורת הפקודה (מלבד שם הקובץ המורץ):

`./my_cp <source_file> <destination_file> <granularity_bytes>`

- `<source_file>`: הנתיב לקובץ המקור שממנו קוראים.
- `<destination_file>`: הנתיב לקובץ היעד שאליו כותבים (אם הקובץ קיים – יש לדרוס אותו; אם אינו קיים – יש ליצור אותו).
- `<granularity_bytes>`: מספר שלם חיובי (int) המציין את גודל הבאפר בבייטים שבו תשתמש התוכנית לכל פעולת read ו-write (למשל: 1, 100, 4096 וכו').

### 3. דרישות טכניות מפורטות למימוש

#### א. טיפול בארגומנטים ובדיקת תקינות

- על התוכנית לוודא שסופק המספר הנכון של ארגומנטים ( $argc == 4$ ). אם לא, יש להציג הודעת שגיאה אינפורמטיבית ל-`stderr` ולסיים את ריצת התוכנית עם קוד שגיאה מתאים (`EXIT_FAILURE`).
- יש להמיר את ארגומנט ה-`granularity` ממחרוזת למספר שלם, ולוודא שהוא ערך חיובי גדול מאפס.

#### ב. הקצאת זיכרון דינמית (Dynamic Memory Allocation)

- אסור להשתמש בבאפר סטטי בגודל קבוע מראש בקוד.
- בהינתן ערך ה-`granularity_bytes`, על התוכנית להקצות את הבאפר באופן דינמי בזיכרון באמצעות `malloc()`.
- חובה לוודא שההקצאה הצליחה, ובסיום ריצת התוכנית לשחרר את הזיכרון באמצעות `free()`.

#### ג. פתיחת קבצים באמצעות קריאות מערכת

- **קובץ המקור:** פתיחה באמצעות קריאת המערכת `open()` עם דגל קריאה בלבד: `O_RDONLY`.
- **קובץ היעד:** פתיחה באמצעות `open()` עם שילוב הדגלים: `O_WRONLY | O_CREAT | O_TRUNC`, בתוספת הרשאות קובץ (למשל 0644).
- חובה לבדוק את ערכי החזרה של `open()` ולטפל בשגיאות באמצעות `perror()`.

#### ד. דרישת מדידת זמן פנימית (Internal Timing Requirement)

מכיוון שהתרגיל בא לבחון את ההשפעה של גודל ה-`Granularity` על ביצועי ה-I/O, על התוכנית למדוד ולהציג בעצמה את הזמן הכולל הדרוש לביצוע כל פעולות הקריאה והכתיבה (ללא זמן פתיחת וסגירת הקבצים).

- **נקודת מדידה ראשונה:** תבוצע מיד לאחר פתיחת קובץ המקור וקובץ היעד בהצלחה, ולפני כניסה ללולאת הקריאה והכתיבה.
- **נקודת מדידה שנייה:** תבוצע מיד עם יציאה מלולאת הקריאה והכתיבה (לאחר שכל הקובץ הועתק), ולפני סגירת הקבצים באמצעות `close()`.
- בסיום המדידה, על התוכנית להדפיס למסך בצורה ברורה את זמן הריצה הכולל (למשל במילישניות או בשניות עם דיוק עשרוני).

#### ה. לולאת I/O (Robust Read/Write Loop)

1. **קריאה:** קרא אל תוך הבאפר הדינמי עד `granularity_bytes` בייטים באמצעות קריאת המערכת `read()`.
2. **טיפול בפסיקות (EINTR):** אם קריאת ה-`read` או ה-`write` נכשלה והחזירה 1- כאשר `errno == EINTR`, יש לנסות שוב באותה איטרציה.
3. **תנאי עצירה:** כאשר `read` מחזירה 0, הגענו לסוף הקובץ (`EOF`), ויש לצאת מהלולאה.
4. **כתיבה חלקית (Partial Writes):** יש לוודא באמצעות לולאה פנימית שכל הבייטים שנקראו אכן נכתבו במלואם לקובץ היעד טרם מעבר למקטע הבא.

#### ו. סגירת משאבים

- בסיום העבודה (הן בהצלחה והן במקרה שגיאה), יש לסגור את הקבצים באמצעות `close()`.

4. מדריך עזר: כיצד למדוד זמן בצורה מדויקת בשפת C  
מכיוון שמדידת זמן אינה נושא שנו בו בהרצאות, להלן הסבר והנחיות כיצד ליישם זאת בקוד שלכם בקלות ובדיוק מרבי באמצעות ספריית הזמן הסטנדרטית של לינוקס (<time.h>).

**הפונקציה המומלצת:** clock\_gettime  
הפונקציה המדויקת והמודרנית ביותר למדידת זמן ריצה של קטע קוד בלינוקס היא clock\_gettime.  
מומלץ להשתמש בשעון הנקרא CLOCK\_MONOTONIC.

- **למה דווקא CLOCK\_MONOTONIC?** שעון זה מייצג את הזמן שעבר מרגע כלשהו בעבר שאינו משתנה, והוא חסין מפני שינויי שעה ידניים במערכת או סנכרון שעונים אוטומטי (NTP). זהו השעון האידיאלי לבדיקות ביצועים (Benchmarking).

**מבנה נתונים ושימוש:**  
הפונקציה מקבלת מצביע למבנה בשם struct timespec, שמכיל שני שדות מרכזיים:

- tv\_sec – שניות (time\_t tipo).
- tv\_nsec – ננו-שניות (טיפוס long, מייצג את השברים של השניה, מ-0 ועד 999,999,999).

#### דוגמת קוד מומלצת לשילוב בתוכנית:

```
#include <stdio.h>
#include <time.h>

// לאחר פתיחת הקבצים בהצלחה, בתוך main, פונקציית:
struct timespec start_time, end_time;

// 1. מדידת זמן התחלה - לפני תחילת הקריאות והכתיבות;
clock_gettime(CLOCK_MONOTONIC, &start_time);

// ---> (read / write) כאן תרוץ לולאת הקריאה והכתיבה שלכם <---

// 2. מדידת זמן סיום - אחרי סיום כל הקריאות והכתיבות, לפני סגירת הקבצים;
clock_gettime(CLOCK_MONOTONIC, &end_time);

// חישוב ההפרש (בשניות ובננו-שניות)
long seconds = end_time.tv_sec - start_time.tv_sec;
long nanoseconds = end_time.tv_nsec - start_time.tv_nsec;

// טיפול במקרה שבו ננו-שניות הסיום קטנות משניות ההתחלה (השאלה מתבצעת מהשניות)
if (nanoseconds < 0) {
    seconds--;
    nanoseconds += 1000000000L; // הוספת שנייה שלמה בננו-שניות
}

// המרה למילישניות לצורך הצגה נוחה
double total_milliseconds = (seconds * 1000.0) + (nanoseconds / 1000000.0);

printf("I/O operations took: %.3f ms (Granularity used)\n", total_milliseconds);
```

## 5. הנחיות הגשה ודרישות מערכת

- שפת תכנות: C בלבד (C11 או מתקדם יותר).

- קומפילציה: התוכנית חייבת להתקומפלך באמצעות gcc תחת לינוקס 6.14 עם דגלי האזהרה המחמירים הבאים, ללא שגיאות וללא אזהרות:

Bash

```
gcc -Wall -Wextra -std=gnu11 -O2 my_cp.c -o my_cp
```

- יש להגיש קובץ קוד מקור יחיד (my\_cp.c) וקובץ Makefile המבצע את הידור התוכנית בצורה אוטומטית.

## דרישות תיעוד והגשה:

1. תיעוד קוד (Comments):

- חובה לכלול הערות (Comments) מקיפות בתוך קוד המקור (C).
- ההערות צריכות לתאר את הלוגיקה של חלקים מורכבים, להסביר את השימוש בכל System Call, ואת משמעות הערכים המוחזרים.

2. מבנה הגשה (GitHub):

- יש ליצור Repository חדש ב-GitHub.
- הרשאות: ודאו כי ה-Repository הוא ציבורי (Public) או שהרשאות הגישה מאפשרות לכל אדם שיש ברשותו הקישור (הבודק) לגשת לקבצים.
- קבצים להעלאה:
  - קובץ קוד המקור
  - Makefile (אם יש).
  - קובץ הרצה (executable).
- קישור: יש להוסיף את הקישור המלא ל-Repository למסך הנלווה.

3. מסמך נלווה (Design Document): חובה להגיש מסמך המתאר לפרטי פרטים את התכנון ואת הלוגיקה של התוכנית.

תוכן נדרש:

- תיאור המטרה ודרישות: תיאור קצר של מטרת התוכנית וכיצד היא עומדת בדרישות.
- ניתוח System Calls: פירוט קריאות המערכת שבהן השתמשתם וכיצד כל קריאה (במיוחד open()) מומשה בהתאם ל-Flags הנדרשים (למשל: O\_RDONLY, O\_WRONLY או O\_CREAT או O\_TRUNC).
- לוגיקת זרימה (Flow): תרשים זרימה (Flowchart) או תיאור מילולי מפורט של שלבי התוכנית (קריאת ארגומנטים, בדיקת קובץ יעד, לולאת העתקה, סגירת קבצים וכו'). וכן כמובן גם טיפול בשגיאות.
- טיפול בשגיאות: פירוט כיצד התוכנית מזהה ומטפלת בשגיאות System Calls.

את המסמך יש להעלות לאתר הקורס.